Still Needed, Hard to Transfer: Using and Reusing Contextual Skills

Anonymous Author(s)

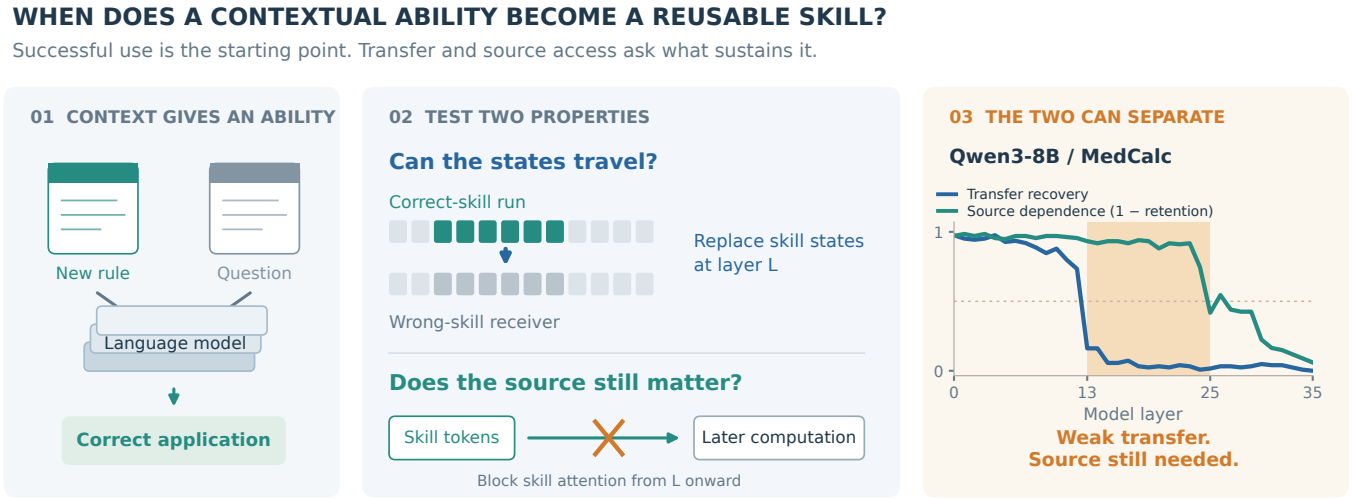


Figure 1: From successful skill use to internal skill reuse. We transplant correct-skill states into a wrong-skill execution and, separately, block subsequent skill attention in the original execution. The right panel shows measured curves on the same 134 Qwen3-8B/MedCalc items: transfer recovery falls before source dependence, leaving an interval in which the skill states transfer poorly while the source still matters. Source dependence is $1 - q(L)$, the fraction no longer solved after attention knockout.

Abstract

Agent skills equip Web agents with task-specific capabilities by loading instructions and supporting resources into context. When a language model successfully uses a skill, can its internal states be extracted and reused, or does that ability remain tied to the original skill context and computation? We investigate this question through two interventions along model depth. We transplant correct-skill span states into a wrong-skill execution and measure recovered performance; separately, we block subsequent attention to the skill in the original execution and measure retained performance. Across three models from two families on MedCalc and TheoremQA, skill-state transfer weakens before dependence on skill access subsides, including on common eligible items. In this intermediate interval, the skill remains important even though its states have little corrective effect in the receiver. Content, position and cross-question controls establish that early transfer carries structured skill information, while repeated late replacement does not restore its effect. Focused experiments on Qwen3-8B/MedCalc show that repairing source states together with their attention caches raises layer-16 accuracy from 3.5% to 42.1%, yet blocking subsequent source reading reduces it to 7.0%. Late transfer therefore depends on computational history, and recovering transfer need not remove source dependence. These findings separate using a contextual skill from reusing its internal states: successful application does not imply an independently reusable internal skill, and failed state transfer does not mean that the model has finished using the source.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Knowledge representation and reasoning*.

Keywords

agent skills, Web agents, large language models, in-context learning, activation patching, context reuse, mechanistic interpretability

1 Introduction

Agent skills are becoming an important source of capability for Web agents. We use *skill* in Anthropic's Agent Skills sense: a package of task instructions, scripts and supporting resources, organized around a `SKILL.md` entry point that an agent discovers and loads when relevant [36]. The Web supplies procedural material for such skills—site instructions, worked examples, API documentation and step-by-step guides—that can enter an agent's context without changing its weights. Web retrieval brings external content into context [28], while Web-agent systems retrieve reusable workflows from past trajectories [32] and combine action programs with

WWW '27, Dublin, Ireland
2027.

natural-language skill guidance [31]. Skills therefore turn external expertise into an operational source of agent capability. Understanding what the model acquires when it reads a skill is central to making that capability reusable.

When a model successfully follows a newly supplied skill, what ability has it acquired? Has the context provided an internal skill that can be extracted and reused, or an ability sustained by the original instructions and computation? This question matters as agents retrieve, share and reuse skills across tasks: successful execution demonstrates that a skill can be used, but does not establish how independently its internal representation can operate. We study this distinction through the instruction content loaded from skills, asking how the ability to reuse their states relates to the ongoing need to access their source.

A natural way to test an internal skill is to move its states to another execution and ask whether they recover the ability. Task and function vectors show that such transfer can work [2, 5, 13, 29]. If a transferable state carries the skill's contribution, losing its effect might suggest that the model has finished using the source. But another possibility is more consequential: the source may still be needed, while its states no longer work in a different computation. We distinguish these possibilities by asking two questions together: *can the skill states correct another execution, and does the original execution still need access to the skill?* Figure 1 illustrates the tests and their separation.

Prior work demonstrates transferable task states, layer-wise subtask scheduling and composable skill caches [13, 16, 35]. We ask a complementary question: *when a source remains necessary, how far can its states be reused, and what restores reuse when it fails?*

We answer these questions with complementary interventions along model depth. State replacement transplants the correct skill's span states into an equal-length wrong-skill receiver and measures recovered task performance. Cumulative attention knockout blocks subsequent attention to the skill in the original correct-skill run and measures retained performance. An effective transfer probe is essential: on a screened clinical-calculator subset, early source-span replacement recovers 0.89 of the correct skill's gain, whereas final-position replacement has little effect. Content, position and cross-question controls establish that this recovery carries structured skill content.

Our main finding is a clear separation between *using* and *reusing* a contextual skill. On Qwen3-8B and MedCalc, transfer drops below half of the available gain at layer 13 of 36, while subsequent skill attention remains important until layer 25. Between these boundaries, the model still needs the source, but transplanting its states has little corrective effect in the receiver. **A skill can remain useful to its original computation after its states cease to be effectively reusable.** This ordering recurs across all six tested combinations of three models and two task domains, including evaluation on common eligible items. Repeating late replacement does not restore early transfer. The results challenge a single progression from reading a skill to possessing an independently reusable internal version: source use and state reuse follow different depth profiles.

The separation also exposes a role for computation history. On Qwen3-8B and MedCalc, replacing both source states and their attention caches raises layer-16 accuracy from 3.5% to 42.1%. Cutting subsequent reader-to-source attention reduces it to 7.0%. Separate path cuts confirm that late dependence reflects continued reading rather than only source-internal updates (§4.4). Thus a useful skill state can require both compatible cached history and continued access to the source.

We make three contributions:

(1) **A validated probe of internal skill reuse.** Source-span interventions recover the correct skill's benefit, while matched content, position and cross-question controls identify the structured skill information responsible for recovery (§4.2).
(2) **A depth-wise separation between skill transfer and source use.** Paired transfer and attention-knockout sweeps reveal an intermediate interval in which skill states transfer poorly even though the original computation continues to depend on the source (§4.3).
(3) **A causal test of computational history and continued reading.** Joint source-state and cache repair recovers substantial late transfer, while reader recutting removes most of that recovery at layer 16. Path-specific cuts distinguish continued source reading from source-internal updates, explaining why useful skill information need not be independently reusable (§4.4).

2 Related Work

Transfer interventions and source dependence. Task and function vectors transfer behaviours from demonstrations [13, 29]. Natural-language instructions can yield different function-vector components [5], and instruction-vector patches can interact superadditively across layers [2]. Transfer also depends on time and semantic scope [18]. These results motivate testing continued source dependence separately: Sia et al. [25] use cumulative context masking, and Pola and Balasubramanian [24] compare instruction-span and final-position patches. Yang et al. [35] combine layer-from context masking with cross-task patching to study subtask scheduling, reporting best-layer transfer strength. Our target is the depth interval where source-span transfer has weakened but cumulative skill-attention knockout still damages the original computation, under matched task and model conditions.

What a transferred state measures. A multiple-choice answer symbol [33] and states from completed reasoning [20] differ from the skill context supplied before generation. Sequential patching [6] and repeated activation steering [19, 23, 26] also intervene beyond our one-time prefill protocol. Following activation-patching methodology [12, 37], we use matched controls, generated answers and identity checks to delimit what recovery establishes. Our depth comparison extends beyond tracing factual recall [8, 21] to the sustained use of skills. Patching effects can depend on interactions with other mediators [30]. Li [16] locate cached conclusions downstream of source fields and show that precompiled skill caches can be repositioned and composed with a target context. These findings establish that execution history matters and that rich cached skills can be reusable. We identify how reuse and source use separate *across depth*: a source can remain necessary after residual transfer weakens, joint source-state/cache repair restores part of that effect, and recutting reader access tests whether the recovered ability

still relies on the source. This connects cache compatibility to the ongoing computation that makes a contextual skill useful.

Skills and compact context representations. Agent Skills package instructions and supporting resources for on-demand use [36]. Skill benchmarks evaluate retrieval and application [10, 17, 27]. Web-agent work supplies reusable workflows and skills to support browser tasks [31, 32]. Such gains depend on evaluation conditions: under a matched total token budget, online skill and memory modules can lose their aggregate advantage [9]. We hold skill selection fixed and study how injected content is used, rather than proposing a skill-learning or retrieval system. Learned compression uses aggregated activations [1], summary tokens [22], compact states [7] or weight updates [3, 4]. We intervene on naturally occurring states without training a compressor.

3 Problem Formulation

Skill context and intervention questions. Our unit of analysis is a skill's instruction body once loaded into model context. We supply this content directly, holding skill selection fixed, and isolate how the model uses and reuses it internally.
State transfer tests recovery in a wrong-skill receiver. Attention knockout tests source dependence in the correct-skill run. Both score generated answers while varying intervention depth.

Skill-span state transfer. Let x be a question, s its correct skill and $\tilde{s}$ an unrelated, equal-length skill. For hidden width D, let H^L_{skill} and H^L_{wrong} denote residual-stream states at layer L under these two contexts. Across the skill's m token positions S, the content-specific difference is a matrix $d^L_S = H^L_{\text{skill}}(S) - H^L_{\text{wrong}}(S) \in \mathbb{R}^{m\times D}$. We use a run containing the wrong skill as the *receiver*, and the correct-skill run as the *donor*. Injection at dose α sets

$$H^L_{\text{patched}}(S) = H^L_{\text{wrong}}(S) + \alpha d^L_S. \tag{1}$$

At $\alpha = 1$, the donor's states replace the receiver's across the span. Other states remain unchanged at that layer. Computation then continues. Matching lengths preserves positions and their encodings. Span interventions change many more states than final-position interventions, which limits direct comparisons. Here the final position is the prompt's end, after the query. State patches are applied during prefill, after which decoding proceeds without further patches. The intervention replaces a block's output, not the complete execution state. Key/value caches already computed at that block and earlier blocks retain their receiver values. Later generated tokens can still attend to these caches, so failed residual-state transfer does not by itself establish failure of a cache-complete source transplant.

Behavioural recovery and evaluation subsets. We score generated answers and measure the fraction of the donor's accuracy gain recovered by an intervention:

$$\rho(\text{arm}) = \frac{\text{acc}(\text{arm}) - \text{acc}(\text{receiver})}{\text{acc}(\text{donor}) - \text{acc}(\text{receiver})}. \tag{2}$$

Here, an arm is an intervention condition, and the donor baseline uses the correct skill in context. Thus $\rho = 1$ recovers the donor's observed gain, and values above one indicate greater accuracy than the donor. We distinguish rescued items (R: wrong without the skill, correct with it), persistent failures (F: wrong under both), kept items (K: correct under both), and broken items (B: correct without, wrong with). Main causal comparisons use rescued items. For span interventions, we further retain only skill groups for which the wrong-skill receiver solves none of the sampled items. We apply this screen separately within each run, model and task, without using intervention outcomes. The estimates are conditional on baseline failure in the sampled group, rather than recovery over the full benchmark.

Measuring source-attention dependence. Cumulative attention knockout masks the skill positions as attention keys at every layer from L onward, during both prefill and decoding. On the knockout run's rescued items within the screened groups, let $q(L)$ be the fraction still solved after this intervention. A low $q(L)$ means that removing subsequent access is damaging. The mask applies to skill keys for all query positions, including skill queries during prefill. It therefore jointly intervenes on reading and source-state updates, rather than isolating a reader-to-source pathway. The transfer boundary is the first layer with $\rho(L) < 0.5$ after early successful transfer. The reading boundary is the first with $q(L) \geq 0.5$. The two boundaries define three operational depth intervals: early transfer, weak transfer with source-attention dependence, and later tolerance to blocking access. The middle interval has $\rho < 0.5$ and $q < 0.5$; later recrossings are reported rather than forced into a monotone stage model. We also repeat both curves on the intersection of transfer items and the knockout run's rescued items within screened groups, retaining each intervention's own baselines. These are summaries of two curves with different intervention backgrounds, not a common measure of information content.

4 Experiments

4.1 Experimental setup

Tasks and models. Our main evaluation pairs MedCalc-Bench [15] with skills from SRA-Bench [27], covering 55 clinical calculators with 20 questions each (1,100 items). Qwen3-8B [34] generates step-by-step reasoning using greedy decoding, without tools. We evaluate its answers with the benchmark scorer. Accuracy is 74.2% with the correct skill, 34.5% without a skill, and 31.5% with a wrong skill. We also test Qwen3-0.6B and Mistral-7B-Instruct-v0.3 [14], and evaluate transfer to SRA-Bench's TheoremQA subset (747 questions and 320 skills) using its prompt and scorer.

Sampling and uncertainty. The main MedCalc span experiments cover 467 rescued items from 48 calculators with at least two such items. Screening for groups whose receiver solves no sampled item retains 135 items from 14 calculators. The main depth sweep and span-quarter comparisons use this same subset. TheoremQA uses the same screen, becoming item-level when a skill has only one question. Intervals resample skill groups, with donor and receiver baselines matched to each intervention's available items. Only complete reruns enter curves. Appendix B records earlier development samples.

Identity control. Writing the receiver's own states back should leave its output unchanged. This identity control exposed a mismatch: capture omitted an attention mask that decoding supplied, changing 108/1,766 Qwen3-8B outcomes (6.1%). Matching the

masks restores exact agreement in every corrected run, for example on all 467 MedCalc items. All main-text interventions use the correction. Appendix A.2 records provenance and Appendix A.1 implementation details. Small numerical discrepancies can propagate through long generations [11].

4.2 Validating a probe of reusable skill content

Changing the intervention positions restores recovery. We next apply Eq. 1 across the skill's own positions. At layer 8, span injection raises accuracy from 0.000 to 0.815 on the screened 135-item subset, recovering 0.89 of the donor's gain (Table 1). Figure 2 compares this with writing donor states at the final position, the final 16 positions, or up to 256 positions after the skill, using the same layer, items and receiver. All positions outside the chosen intervention retain their receiver states. On the same items, the last 1, 16 and up to 256 positions recover 0.03, 0.06 and 0.05, respectively.

Dose, transfer, order and noise controls. Recovery changes nonlinearly with injection strength: $\alpha = 0.5, 0.6, 1.0$ gives 0.04, 0.53, 0.89 recovery, while doubling the dose reduces it to 0.77. To test content specificity, we compare correct and other-skill injections over a common prefix length: the correct prefix recovers 0.65, whereas content from an unrelated skill recovers 0.05. A same-family mean control recovers 0.12 on its 41 eligible items. Permuting positions also gives 0.05, norm-matched Gaussian noise gives 0.02, and the identity control gives 0.00. These controls show that recovery depends on the injected content and its arrangement across positions, rather than perturbation size alone.

These results are conditional on the receiver screen. On excluded groups, other-skill and noise controls perform better, limiting extrapolation to the full benchmark.

Does the transferred information generalize across questions? A successful same-question patch could carry a task-specific answer rather than reusable skill content. Table 2 tests this alternative by varying the injected section and the donor question. Of four contiguous span quarters, the formula quarter gives the largest standalone recovery (0.56), compared with 0.89 for the full span. With the skill before the question, causal masking prevents its states from depending on later question tokens: states are identical across instances at equal prompt length and nearly identical otherwise (cosine ≥ 0.994 over all 48 calculators). Placing the skill after the question allows such dependence. Even then, states from other instances of the same skill recover 0.98 at layer 8, close to the current instance's 1.02. At layer 16, the gap widens to 0.29 versus 0.49, while the instance-specific remainder recovers little on its own. Early transfer therefore relies largely on content shared across questions, with greater question dependence later.

Two completed corrected controls test whether a shared shift or a change in state norm could explain recovery. Averaging content differences across skills recovers only 0.03 with the skill first ($n = 135$) and 0.04 with it last ($n = 151$). Rescaling patched states to match the receiver's norm at each position preserves 0.87 recovery, compared with 0.89 without rescaling. In the corrected rerun, layer-16 recovery is 0.49 with the skill last (151 items, 17 calculators) and 0.06 with it first (135 items, 14 calculators). These differently screened subsets do not identify a prompt-order effect.

4.3 Transfer fails before source dependence ends

The effective early span intervention gives us a probe with which to test the paradox: does losing the transfer effect at later layers mean the model no longer needs the skill? Figure 3 tests this inference directly.

Transfer weakens while reading remains important. Figure 3 compares transfer through span replacement with continued use through attention across all six model–task combinations. We first examine Qwen3-8B on MedCalc. Every layer is measured, using the same screened groups and each intervention's eligible items. Replacement recovers 0.92–0.98 through layer 7, 0.89 at 8 and 0.74 at 12 of 36, then 0.16 at 13 and at most 0.07 from 15 onward ($n = 135$, 14 calculators). We define the crossing below $\rho = 0.5$ as the *transfer boundary* (reported as transfer in Table 4). The log-probability of the donor's generated reasoning also drops between these layers, suggesting that binary answer grading sharpens, but does not fully explain, the transition (Appendix B). To test continued reading, *cumulative attention knockout* blocks attention to skill tokens at every layer from L onward. On the 134 rescued items of the same subset, 0.03–0.12 stay correct for $L \leq 23$. This rises to 0.25 at 24, first passes half at 25 (0.58) and reaches 0.94 at 35. The transfer boundary therefore lies at $13/36 = 0.36$ of depth and the reading boundary at $25/36 = 0.69$, using the same screen and restricting knockout to its donor-correct items. Writing the last prompt position instead recovers at most 0.06 at any layer. **The same source is hard to reuse in the receiving computation while remaining useful to the ongoing correct computation.** Continued dependence on skill attention thus outlasts successful transfer from the skill's states. Transfer measures recovery in a wrong-skill receiver, whereas knockout measures dependence in a correct-skill run. The separation identifies the intermediate regime operationally: from layers 13 through 24, span replacement recovers less than half the gain and blocking further reading leaves fewer than half the rescued items correct. The first crossing is at layer 25; retention briefly falls below half again at 26, then rises above it from 27 onward. The boundary summarises the curve rather than asserting a monotonic transition. Thus transfer loss precedes, rather than demonstrates, tolerance to removal of source access.

The ordering survives common-item evaluation. The six depth sweeps already cover Qwen3-8B, Qwen3-0.6B and Mistral-7B on MedCalc and TheoremQA. Their original eligibility criteria retain different numbers of items for transfer and knockout. We therefore recompute both curves on their intersection, without selecting on intervention outcomes. All six first-crossing pairs remain unchanged (Table 3). Skill-group bootstrap 95% intervals for the reading-minus-transfer boundary difference exclude zero in every combination. This supports the ordering beyond the Qwen family and on common items. The ordering also persists at thresholds 0.4 and 0.6.

Repeated replacement does not bridge the gap. To test whether late failure reflects intervening at too few layers, we replace span states repeatedly within consecutive layer windows, using the battery's receiver and the same 135 items as the depth sweep. Windows 8–11,

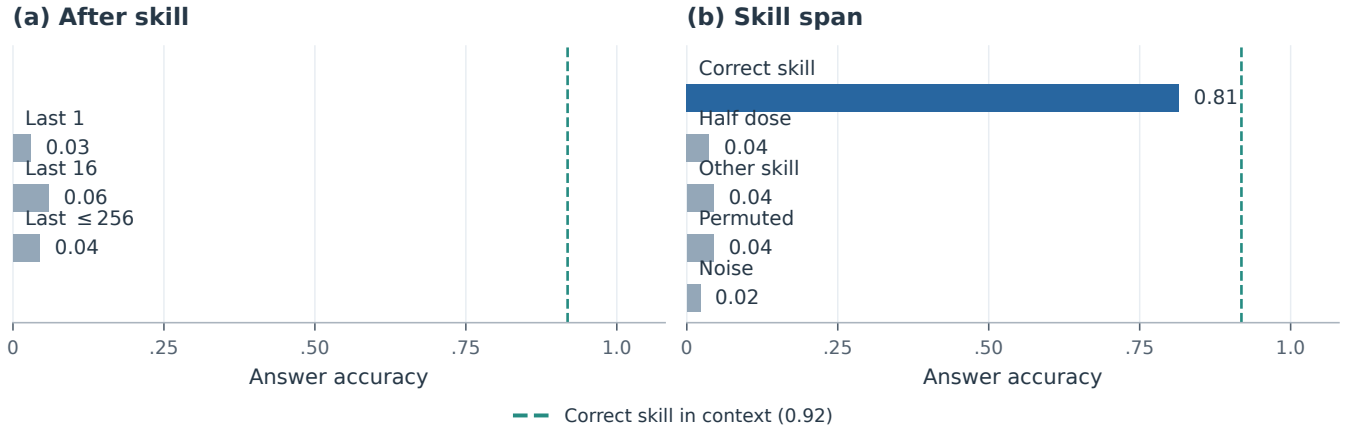


Figure 2: Matched position comparison on MedCalc with Qwen3-8B. Both panels use layer 8, the same wrong-skill receiver and 135 items from 14 calculators. Left: donor states written at the last 1, 16, or up to 256 available positions after the skill. Right: interventions over the skill span. The dashed line marks correct-skill accuracy; receiver accuracy is zero. All marks show point estimates.

Table 1: Span interventions on the screened MedCalc subset: 135 items from 14 calculators, selected from 467 rescued items across 48 calculators by requiring the wrong-skill receiver to solve no sampled item in a group. The same-family control is available on 41 items. Each row uses baselines matched to its own items in Eq. 2. Entries show point estimates.

Intervention	n	Accuracy	Recovery ρ
correct skill in context	135	0.919	+1.00
$h_{\text{recv}} + d$ (full span)	135	0.815	+0.89
correct content, shared prefix	135	0.593	+0.65
half dose ($\alpha = 0.5$)	135	0.037	+0.04
another skill (individual donor)	135	0.044	+0.05
mean from another skill family	135	0.044	+0.05
mean from the same skill family	41	0.122	+0.12
positions permuted	135	0.044	+0.05
norm-matched Gaussian noise	135	0.022	+0.02
identity intervention	135	0.000	+0.00
wrong skill, unpatched	135	0.000	+0.00

12–15, 14–19 and 16–35 recover 0.89, 0.74, 0.16 and 0.06. The last equals layer 16 alone. Writing every layer at once recovers 0.98, so the transplant itself remains effective when it starts early enough. The identity control is exact on all 467 items. On TheoremQA (99 items, 75 groups, with its battery's receiver), the same windows recover 0.84, 0.61, 0.38, 0.20, the last again equal to layer 16 alone. Repeated late injection therefore does not restore early-layer recovery in these tests. This rules out a simple shortage-of-interventions explanation for the late transfer failure in these tests.

Replication across models and task domains. We extend span injection and the depth comparisons to Qwen3-0.6B ($D = 1024$), Mistral-7B ($D = 4096$, 32 layers), and TheoremQA. Each model is injected at an early layer, and the receiver screen is applied separately to each run. Table 4 collects the transfer and reading boundaries alongside early-transfer and content-control results. The two boundaries use the same screened groups but separately eligible items, so their ordering is a group-level comparison rather than a strictly paired per-item estimate; the common-item check in Table 3 confirms the same ordering.

Span recovery ranges from 0.74 to 0.94 across the six model–task combinations. The largest control is 0.35, for a same-family mean on 31 Mistral/MedCalc items, where full-span recovery on the same items is 0.71. In every model–task combination, the transfer boundary occurs before the reading boundary. The measured transfer boundaries lie at 0.33–0.39 of model depth within Qwen and at 0.19–0.22 in Mistral. Reading boundaries also vary by task: 0.58–0.71 on TheoremQA and 0.69–0.84 on MedCalc across the three models.

4.4 Testing why late skill states fail to transfer

The depth gap raises a computation-history question. Replacing the source-span residual leaves its previously computed attention

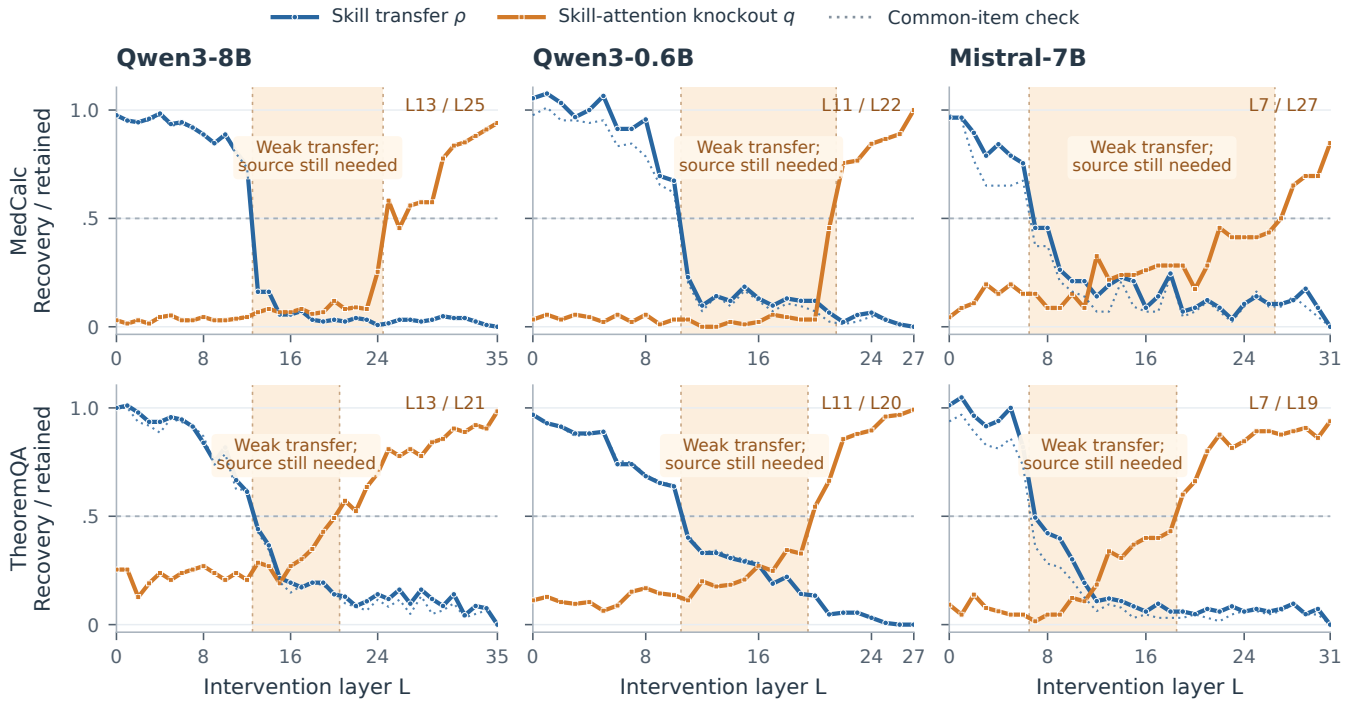


Figure 3: Skill transfer and reading dependence across three models, two families and two tasks. Every layer is measured; the horizontal axis gives the actual intervention layer L. Blue curves report span-transfer recovery ρ, and orange curves report the retained fraction q under cumulative skill-attention knockout from that layer onward. The shaded middle interval lies between the first half-effect boundaries, annotated as transfer / reading layers. Thin dotted curves repeat the comparison on identical eligible item IDs (134, 90, 46 for MedCalc; 63, 125, 65 for TheoremQA, in column order). At the orange interval's left boundary, transfer becomes weak in the wrong-skill receiver, yet removing skill attention still strongly harms the correct-skill run. Table 3 reports the common-item boundaries.

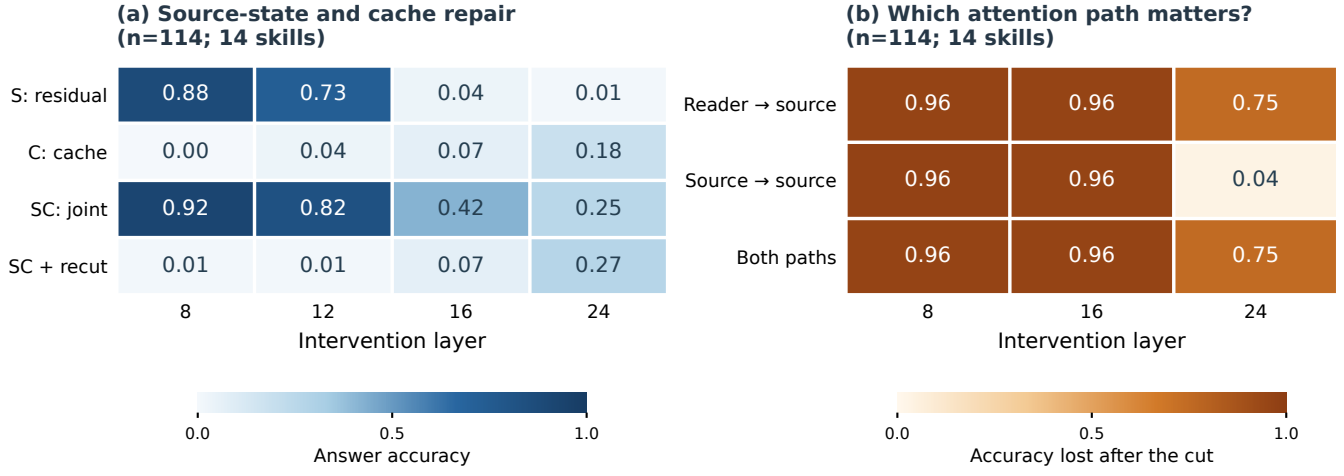


Figure 4: Source history repairs transfer; continued reading supports its use. Qwen3-8B on the same 114 rescued MedCalc items from 14 calculators. Left: answer accuracy after residual (S), source-cache (C), joint (SC), and joint-plus-reader-recut interventions. SC pairs source states with their cached history; recutting begins at the next layer. Right: fraction of correct donor answers lost after reader-to-source, source-to-source, or combined cuts, during both prefill and decoding from the labelled layer. All cells are measured point estimates.

Table 2: Content and question dependence of span states, Qwen3-8B. Top: each contiguous quarter is injected separately, on 135 items from 14 calculators. Bottom: with the skill after the question, content shared across other instances and the instance-specific remainder are injected separately, on 151 items from 17 calculators. Each comparison uses its screened subset from the larger rerun. All entries report recovery ρ.

Skill quarters, layer 8, skill before question ($n = 135$)	
Injected content	Recovery ρ
Prose description	0.09
Formulas	0.56
Tool signatures	0.12
Worked example	0.05
Full span	0.89

Question dependence, skill after question ($n = 151$)		
Injected content	Layer 8	Layer 16
This instance	1.02	0.49
Other instances of the same skill	0.98	0.29
Instance-specific remainder	0.03	0.05

Table 3: Common-item depth validation. n is the intersection size. T/R gives first transfer / reading boundary layers; $\Delta = R - T$. Entries show point estimates.

Task	Model	n	T/R	Δ
MedCalc	Qwen3-8B	134	13/25	12
MedCalc	Qwen3-0.6B	90	11/22	11
MedCalc	Mistral-7B	46	7/27	20
TheoremQA	Qwen3-8B	63	13/21	8
TheoremQA	Qwen3-0.6B	125	11/20	9
TheoremQA	Mistral-7B	65	7/19	12

caches and reader history unchanged. We test whether repairing the source cache restores transfer, and whether the restored effect still requires access to the source.

Source-state and cache repair. On Qwen3-8B and MedCalc, we compare source-residual replacement (S), source key/value-cache replacement (C), their combination (SC), and SC followed by blocking reader-to-source attention from the next layer. C replaces source cache entries through the intervention layer; earlier reader history remains fixed. The four conditions are evaluated at layers 8, 12, 16, 24 on a fixed 124-item set from 14 calculators, with fresh no-skill, donor and receiver baselines. Of these, 114 items across all 14 calculators are donor-correct and incorrect under both receiver and no-skill baselines; both panels use these identical items. All 48 own-state controls across the four GPU shards reproduce output tokens exactly. This design tests whether repairing the source's cached contribution improves on residual replacement alone, and whether any recovery uses subsequent source access.

Separate reading from source updates. A second comparison blocks only reader-to-source attention, only source-to-source attention, or both, from layers 8, 16, 24 onward during prefill and decoding. Separating these paths identifies which component of the original cumulative knockout produces the performance loss.

Repairing history restores substantial late transfer. At layer 16, S alone solves 4/114 items (3.5%), C solves 8/114 (7.0%), and SC solves 48/114 (42.1%). The prespecified SC-minus-S contrast is 38.6 percentage points, and the joint gain exceeds the sum of the two individual gains by 31.6 points. Source states and their cached history therefore contribute jointly: weak residual transfer does not mean that useful source information has disappeared. At layer 24, SC still improves on S (25.4% versus 0.9%), but no longer clearly exceeds C alone (18.4%). The repair is substantial, rather than complete.

Recovered transfer still relies on source reading. After SC at layer 16, cutting reader-to-source attention from the next layer reduces accuracy from 42.1% to 7.0%. In the unmodified donor, reader-only and combined cuts at layer 24 both leave 25.4% correct, whereas a source-self-only cut leaves 96.5%. Late source dependence thus reflects continued reading, not merely disruption of the source's internal updates. Earlier, at layers 8 and 16, either path cut is damaging. At layer 24, recutting after SC gives 27.2%, with no detected additional loss; we therefore locate the repaired-yet-reading-dependent effect at layer 16, rather than attributing it to every tested depth.

The 38.6-point layer-16 repair gain remains 38.5 points on the 96 items whose three baselines finish without truncation. Even assigning all truncated S outputs correct and all truncated SC outputs incorrect leaves a 35.1-point gain. Skill-group uncertainty, fixed-set estimates and truncation diagnostics are reported in Appendix B.1.

5 Conclusion

Successfully following a skill does not make its internal states an independently reusable substitute for the source. Our experiments reveal a depth interval in which skill-state transfer has weakened while the original computation continues to depend on skill attention. This separation recurs across three models and two task domains, including common-item evaluation. The model can still use a skill after its states have become ineffective at correcting another execution.

Content and cross-question controls show that effective early transfer carries structured skill information. Matching state norms preserves that effect, while repeated late replacement does not restore it. Source-cache repair further shows that late residual-transfer failure is partly reversible: pairing states with their cached history raises layer-16 accuracy from 3.5% to 42.1%. Yet cutting subsequent source reading reduces it to 7.0%. Separating attention paths confirms that late source dependence is driven by continued reading. Useful skill information can therefore survive a failed residual transplant while requiring a richer execution state and further access to its source.

Contextual competence therefore has two distinct properties: the ability to support the current computation and the ability to be reused in another. For skill-sharing agents and context-reuse systems, both need to be tested. A representation that works within

Table 4: Replication with the corrected implementation. Injection layers are 8 for Qwen3-8B, 6 for Qwen3-0.6B and 4 for Mistral-7B. All entries are point estimates. Control reports the maximum over half-dose, other-skill, permutation and noise comparisons, including same-family means available on 41 MedCalc/Qwen3-8B and 31 MedCalc/Mistral items. n_T/n_R gives the battery and knockout sample sizes; groups refer to the battery. Transfer and reading report the first boundary layers at one-layer resolution. Reading is the first layer from which blocking attention to the skill still leaves at least half of the rescued items correct, measured on the knockout run's rescued items within the same screened groups.

Task	Model	n_T/n_R (groups)	ρ	Control	Transfer L	Reading L
MedCalc	Qwen3-8B	135/134 (14)	0.89	0.12	13	25
TheoremQA	Qwen3-8B	99/63 (75)	0.84	0.25	13	21
MedCalc	Qwen3-0.6B	362/90 (38)	0.91	0.30	11	22
TheoremQA	Qwen3-0.6B	132/125 (97)	0.74	0.13	11	20
MedCalc	Mistral-7B	92/46 (20)	0.84	0.35	7	27
TheoremQA	Mistral-7B	115/65 (94)	0.94	0.27	7	19

its original computation is not yet a portable replacement for the instructions that produced it.

References

[1] Thibaud Ardoin, Semira Einsele, Evis Bregu, and Gerhard Wunder. 2026. Prompt Compression via Activation Aggregation. *arXiv preprint arXiv:2607.08399* (2026).

[2] Irina Bigoulaeva, Jonas Rohweder, Subhabrata Dutta, and Iryna Gurevych. 2026. Patches of Nonlinearity: Instruction Vectors in Large Language Models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 12209–12262. doi:10.18653/v1/2026.acl-long.559

[3] Lucas Caccia, Alan Ansell, Edoardo Ponti, Ivan Vulić, and Alessandro Sordoni. 2025. Training Plug-n-Play Knowledge Modules with Deep Context Distillation. In *Conference on Language Modeling (COLM)*. arXiv:2503.08727.

[4] Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to Instantly Internalize Contexts. *arXiv preprint arXiv:2602.15902* (2026).

[5] Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. 2025. Do different prompting methods yield a common task representation in language models? *arXiv preprint arXiv:2505.12075* (2025). https://arxiv.org/abs/2505.12075

[6] Murat Dura, Serkan Öztürk, and Selma Tekir. 2026. Mechanistic Interpretability of Chain-of-Thought Reasoning via Sequential Activation Patching. *arXiv preprint arXiv:2608.22332* (2026).

[7] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context Autoencoder for Context Compression in a Large Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2307.06945.

[8] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. 2023. Dissecting Recall of Factual Associations in Auto-Regressive Language Models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. arXiv:2304.14767.

[9] Sina Hajimiri, Masih Aminbeidokhti, Jose Dolz, Ismail Ben Ayed, Issam H. Laradji, Spandana Gella, and Nicolas Gontier. 2026. Are Online Skill and Memory Modules Always Worth Their Tokens? A Budget-Constrained Study of Web Agents. *arXiv preprint arXiv:2606.15017* (2026). https://arxiv.org/abs/2606.15017

[10] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? *arXiv preprint arXiv:2603.15401* (2026).

[11] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[12] Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255* (2024).

[13] Roee Hendel, Mor Geva, and Amir Globerson. 2023. In-Context Learning Creates Task Vectors. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. arXiv:2310.15916.

[14] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[15] Nikhil Khandekar, Qiao Jin, Guangzhi Xiong, Soren Dunn, Serina S. Applebaum, Zain Anwar, Maame Sarfo-Gyamfi, Conrad W. Safranek, Abid A. Anwar, Andrew Zhang, Aidan Gilson, Maxwell B. Singer, Amisha Dave, Andrew Taylor, Aidong Zhang, Qingyu Chen, and Zhiyong Lu. 2024. MedCalc-Bench: Evaluating Large Language Models for Medical Calculations. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2406.12036.

[16] Bojie Li. 2026. Models Take Notes at Prefill: KV Cache Can Be Editable and Composable. *arXiv preprint arXiv:2606.17107* (2026). https://arxiv.org/abs/2606.17107

[17] Xiangyi Li, Yimin Liu, Wenbo Chen, et al. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *arXiv preprint arXiv:2602.12670* (2026).

[18] Yuxuan Li, Declan Campbell, Stephanie C. Y. Chan, and Andrew Kyle Lampinen. 2025. Just-in-time and distributed task representations in language models. *arXiv preprint arXiv:2509.04466* (2025).

[19] Sheng Liu, Haotian Ye, Lei Xing, and James Y. Zou. 2024. In-context Vectors: Making In Context Learning More Effective and Controllable Through Latent Space Steering. In *International Conference on Machine Learning (ICML)*. arXiv:2311.06668.

[20] Houman Mehrafarin, Amit Parekh, and Ioannis Konstas. 2026. When Chain-of-Thought Fails, the Solution Hides in the Hidden States. In *Findings of the Association for Computational Linguistics: EMNLP 2026*. arXiv:2604.23351.

[21] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and Editing Factual Associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2202.05262.

[22] Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to Compress Prompts with Gist Tokens. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2304.08467.

[23] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. 2024. Steering Llama 2 via Contrastive Activation Addition. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2312.06681.

[24] Aditya Pola and Vineeth N. Balasubramanian. 2025. Where does an LLM begin computing an instruction?. In *UniReps Workshop*. extended abstract; arXiv:2511.10694.

[25] Suzanna Sia, David Mueller, and Kevin Duh. 2024. Where does In-context Learning Happen in Large Language Models?. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2403.04510.

[26] Alessandro Stolfo, Vidhisha Balachandran, Safoora Yousefi, Eric Horvitz, and Besmira Nushi. 2025. Improving Instruction-Following in Language Models through Activation Steering. In *International Conference on Learning Representations (ICLR)*. arXiv:2410.12877.

[27] Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill Retrieval Augmentation for Agentic AI. *arXiv preprint arXiv:2604.24594* (2026).

[28] Jiejun Tan, Zhicheng Dou, Wen Wang, Mang Wang, Weipeng Chen, and Ji-Rong Wen. 2025. HtmlRAG: HTML is Better Than Plain Text for Modeling Retrieved Knowledge in RAG Systems. In *Proceedings of the ACM on Web Conference 2025*. Association for Computing Machinery, New York, NY, USA, 1733–1751. doi:10.1145/3696410.3714546

[29] Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. 2024. Function Vectors in Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2310.15213.

[30] Sankaran Vaidyanathan, David Arbour, Aaron Mueller, Scott Niekum, and David Jensen. 2026. The Curse of Multiple Mediators: Hidden Interaction Effects in Activation Patching. *arXiv preprint arXiv:2606.27510* (2026). https://arxiv.org/abs/2606.27510

[31] Zhaoyang Wang, Qianhui Wu, Xuchao Zhang, Chaoyun Zhang, Wenlin Yao, Fazle Elahi Faisal, Baolin Peng, Si Qin, Suman Nath, Qingwei Lin, Chetan Bansal, Dongmei Zhang, Saravan Rajmohan, Jianfeng Gao, and Huaxiu Yao.

2026. WebXSkill: Skill Learning for Autonomous Web Agents. *arXiv preprint arXiv:2604.13318* (2026). https://arxiv.org/abs/2604.13318

[32] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[33] Sarah Wiegreffe, Oyvind Tafjord, Yonatan Belinkov, Hannaneh Hajishirzi, and Ashish Sabharwal. 2025. Answer, Assemble, Ace: Understanding How LMs Answer Multiple Choice Questions. In *International Conference on Learning Representations (ICLR)*. arXiv:2407.15018, spotlight.

[34] An Yang, Anfeng Li, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[35] Zhipeng Yang, Junzhuo Li, Siyu Xia, and Xuming Hu. 2025. Internal Chain-of-Thought: Empirical Evidence for Layer-wise Subtask Scheduling in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 22536–22564. doi:10.18653/v1/2025.emnlp-main.1147

[36] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills

[37] Fred Zhang and Neel Nanda. 2024. Towards Best Practices of Activation Patching in Language Models: Metrics and Methods. In *International Conference on Learning Representations (ICLR)*. arXiv:2309.16042.

A Setup, in full

Two task families. Real skills, the main material: MedCalc-Bench instances with the 55 gold calculator skills released with SRA-Bench—55 calculators × 20 instances = 1,100 items, graded deterministically against a numeric tolerance with no model in the loop. Skills and tasks come from released benchmarks rather than being designed to favour the intervention. Qwen3-8B, single turn, no tools, greedy. Behavioural baseline and replication intervals resample skill groups because items sharing a skill are dependent; original item-level intervals in individual tables and plots are identified separately in the main text. *A synthetic positive control*: a fictional unit system with a one-page document giving its conversion table, so we control the mapping and can measure the no-skill baseline directly. Items are generated from the table ($n = 358$), and every item whose answer appears in a worked example is dropped.

Two dependent variables. Accuracy, from a greedily decoded free-form answer graded by the task's own scorer, and the teacher-forced log-probability of the gold answer. They fail differently and disagree often enough that reporting one alone is unsafe; accuracy is read first throughout. No item was rewritten into multiple choice to obtain an accuracy channel.

Interpreting recovery. Equation 2 normalises an intervention by donor and receiver accuracy on the same items. It measures the fraction of the available gain, not raw accuracy or a percentage of information transferred. Values can exceed one. Comparisons across protocols must retain their own paired baselines.

Focused mechanism protocol. The Qwen3-8B/MedCalc mechanism experiments use BF16, eager attention and a 1,536-token generation limit. A bridge on one fixed item from each of 14 calculators compares full-prompt prefill with prefill split before the final prompt token. All 84 own-state checks reproduce their corresponding receiver token sequences exactly. Changing the prefill protocol changes one donor's and one receiver's answer correctness. The formal repair and path comparisons therefore use the same split protocol and fresh no-skill, donor and receiver baselines. Truncation counts and sensitivity analyses are retained with the result records.

The instrument's own noise floor, measured. Every decoding run carries a *null* arm: the receiver's own span states are captured and written back over themselves. With a length-matched receiver this is a bit-for-bit identity, so the null arm must equal the unpatched receiver at every layer. Before the fix of Appendix A.1 it did not, quite. On Qwen3-8B, across 1,766 paired observations from every pre-fix run that carries both arms, the null arm agrees with the receiver on 93.9% of items: 108 flip, 46 from wrong to right and 62 from right to wrong, symmetric within a sign test ($p = 0.15$). The pipeline itself is deterministic—the same configuration run on two different machines agreed on all 360 item-by-arm cells—so this is not run-to-run variation but a reproducible property of the instrument: the capture and decoding passes took different attention kernels, which perturbs the written-back state by 1.2% at layer 8, and the dependent variable is the end of a chain of thought several hundred tokens long, which is long enough for that to change an argmax somewhere and propagate. It scales with the model: the same arm flips 0.6% of items on Qwen3-0.6B (1,696 pairs) and 0.8% on Qwen3-1.7B (1,338), against 6.1% on the 8B. After the fix it is exact (80/80).

Three consequences are used throughout. First, **an effect smaller than about six points is not resolvable on the 8B** in pre-fix runs, whatever a binomial interval says. Second, the floor is not uniform: on the persistent-failure cell it is 16%, between two and three times the average, which is what one expects of items on a decision boundary, so numbers read on that cell are read against a coarser ruler. Third, on the restricted split the pre-fix null arm read 0.034 rather than zero—the split conditions on the receiver failing, so a flip there can only go upward—so pre-fix span numbers are read against 0.034; after the fix it reads exactly 0.000, the floor Table 1's 0.815 is read against. We have rarely seen this control reported in the patching literature; at a single answer token it is small enough to be invisible, and it becomes large only once the dependent variable is a long generation.

Four cells. Every item is labelled by what the document did to it: **R** (rescued: wrong without, right with), **F** (persistent: wrong both ways), **K** (kept: right both ways), **B** (broken: right without, wrong with). **R is the only cell in which the document demonstrably works**, every table here is read by cell, since the aggregate over all four can be sign-inverted relative to R.

A.1 Experimental detail

Models and numerics. Qwen3-1.7B (28 layers, $d = 2048$) and Qwen3-8B (36 layers, $d = 4096$); §4.3 adds Qwen3-0.6B (28 layers, $d = 1024$) and Mistral-7B-Instruct-v0.3 (32 layers, $d = 4096$), whose chat template has no system role, so the system text is merged into the user turn. Greedy decoding throughout, `enable_thinking` disabled and verified by a self-test on the rendered template. Attention-mask interventions use eager attention with explicit mask checks; the earlier decode-heavy residual-replacement runs use sdpa, which was about 2.5× faster on a 2k-token prompt in the original setup. The 1.7B synthetic runs are fp32: bf16 costs 0.5–2.5 nats on the log-probability of a token deep in the tail, and every log-probability quantity here is a difference of such numbers. The 8B runs are bf16 and the gaps we report from them are 5–8 nats, well outside that error; the accuracy channel is immune to it either way.

Table 5: Full-dataset rerun against the subset the main text cites, Qwen3-8B on MedCalc, layer 8. Recovery is Eq. 2; entries show point estimates. *per-calc* is the main text's restriction, *per-item* the weaker one. Controls reach 0.12 under the group screen and 0.25 under the item screen; each arm uses its own matched baselines.

arm	full, per-calc (n = 135)	subset (n = 60)	full, per-item (n = 369)
transplant `real`	**0.89**	0.87	0.94
same skill, matched prefix	0.65	0.73	0.84
half dose $\alpha = 0.5$	0.04	0.06	0.11
positions permuted	0.05	0.08	0.21
norm-matched noise	0.02	0.10	0.18
other calculator	0.05	—	0.25
other family, mean	0.05	0.06	0.16
same family, mean	0.12	0.05	0.24
identity patch	0.00	0.00	0.00
rank 128 / 64 / 32 / 16	0.77/0.63/0.12/0.05	0.77/0.54/0.13/0.06	0.88/0.70/0.22/0.18
bottom-64 / bottom-16	0.03/0.04	0.00/0.06	0.11/0.12
quarters $q_0/q_1/q_2/q_3$	0.09/**0.56**/0.12/0.05	0.12/0.56/0.12/0.12	0.18/0.64/0.22/0.19

Self-tests, and one that failed. Three checks run with the experiments. (i) *Self-transplant*: donor and receiver are the same forward, so the patch must be an exact no-op. On the fp32 synthetic runs the deviation is 0.0 nats at every layer. On the bf16 8B runs it is up to 0.38 nats—bf16 arithmetic over two sequence lengths—while the accuracy channel remains an exact no-op at every layer, which is how we tell the two apart. (ii) *Last-layer patch*: replacing the final layer's output at the patched position makes that position's logits the donor's by construction, so the next token must match exactly. (iii) *Query-side control* for the knockouts.

An error worth recording. The first version of the real-skill scoring code patched at index −1 of a forward run over prompt *and* answer, which is the last token of the answer—after every position whose logits are scored. Every arm returned the untouched baseline to six decimals, which is what made it visible. Relative indices are the wrong default whenever the forward length varies between the capture and the patch.

Control documents. The synthetic controls are generated from the skill document by `make_controls.py`, which checks two things: the corrupted document's numbers must not coincide with the task set's gold answers (7 of 249 do, 2.8%), and the shuffle must not be the identity. All three controls are stored under the same filename in separate directories, because the injector derives the "# Skill: *name*" header from the file stem—a file called `SKILL.zorb-units.corrupted.md` would announce the manipulation in the prompt, and the control would then differ from the treatment in two ways.

Item set. The synthetic set is generated from the conversion table with the value range 2–40, dropping any item whose answer also appears in a worked example of the document (n = 358; the sweeps use the first 120 or 200). Distractors are values reachable by misreading the table—the neighbouring row, the wrong family, the inverted direction—never random numbers, and the key is rotated across A/B/C/D so a position-biased model still scores at chance.

A.2 Evaluation scope and intervention provenance

An accidental replication. A queueing mistake ran one 36-layer synthetic sweep (358 items, two arms, generations up to 900 tokens) twice, on two different GPUs in two different containers. All 36 layer files agree byte for byte. This does not bound the nondeterminism discussed in §4.1—it is one configuration on one pair of devices—but it is the only end-to-end check of it we have, and it came out exact.

Sample size and scope. The battery was developed on $n = 40$ (ten calculators × four rescued instances), which gives intervals of roughly ±0.15; §4.2 reruns it on forty calculators and §4.3 on further models and a second task, and the conclusions are the ones that survived that. We report profiles and not individual layers: the one non-monotonicity in the depth curve (0.250 at layer 18 against 0.475 at 20) has overlapping intervals and we do not interpret it. Among the three models, Qwen3-1.7B is the weakest rung: the document rescues only 8–11% of its items, so its truncation curve is not resolved and is reported for completeness rather than relied on. The width argument rests on Qwen3-0.6B against Qwen3-8B, a factor of four within one family; a second family of the same width needs more directions. In the pre-fix development runs, four independent shards recomputed the three baselines and agreed to the item (0.950 / 0.025 / 0.175), and the no-op arm sits on the receiver's baseline at all twelve layers to within the 5% numerical floor of §4.1, so the instrument is checked even where the estimate is coarse. The span channel, the depth sweep and the truncation curve have three models and two tasks; the knockouts two models and two tasks; the final-position results two models (Qwen3-1.7B, fp32); the document-order comparison and the within-document localisation one.

Which items the causal claims are about. Two restrictions stack. Causal claims are made only on the rescued cell, the only cell in which the document demonstrably works; on the persistent-failure cell the same arms track the document rather than correctness—the transplant fails exactly where the document fails—which is

the right behaviour but is not a substitute for the other cells. And within that cell the span arms are read on the calculators whose *receiver* never solves an item, because on the others the model does not need the document, and before the fix any disruption of the wrong one, noise included, let it fall back on what it already knows ($\rho = 0.14$ for noise there against 0.00 here). After the fix noise no longer does that (0.00 on those calculators), but a donor from another skill still reads 0.47 there against 0.19 on the rest. On forty calculators the restriction keeps fifteen of them and 60 items. On this split, accuracy is 0.750 for the transplant and at most 0.083 for the tested negative controls; the main text distinguishes item-level intervals from group-bootstrap uncertainty. Those forty calculators are themselves the 160 rescued items with the most instances per calculator, out of 467. We therefore reran the battery, the truncation curve, the depth sweep, the dose curve and the within-document quarters over *every* rescued item of every calculator with at least two (467 items, 48 calculators; the restriction then keeps 135 items over 14). Every conclusion holds and no control moves above 0.12; Appendix B gives the comparison arm by arm.

What the rank result does and does not establish. Rank truncation analysis truncates the injected content matrix to rank k at a layer where the untruncated version recovers most of the effect, and recovery stays near the control floor at $k \leq 32$ while the bottom-k control stays at the floor. That licenses the forward direction: the effect requires directions of the same order as the span's participation ratio, and within the Qwen family neither number scales with width. It does not license a claim that the number belongs to the document alone (Mistral-7B needs more), nor a converse about depth. Supplying a deep layer with a higher-rank message taken from a shallow one does not restore the handover ($\rho = 0.00$ against 0.06 for the same-layer donor, 0.00 with the norm matched), so we do not claim that what a deep layer lacks is rank.

Diagnostic results for the behavioural readout. We could not find any representational quantity at the final position that distinguishes the items a skill rescues from the ones it does not: the norms and angles agree to three digits, and the one similarity measure that separates them under the gold skill separates them equally under a wrong one. And the document acts through a chain of thought, so every causal measurement decodes several hundred tokens; scoring under a forced `ANSWER:` cue instead leaves almost no behavioural effect (0.178 with the skill, 0.131 without and 0.100 with a wrong one, on 720 items) while retaining a +1.08 nat log-probability difference—so the span channel has no usable log-probability reading, and is reported on accuracy alone.

An instrument fault and result provenance. State capture omitted the attention mask supplied during decoding. Although mathematically equivalent for an unpadded sequence, these calls selected different kernels. Their states differed by 1.2% at layer 8 and 25% at layer 34, and identity patching changed 6.1% of Qwen3-8B answers. Matching the masks restores exact identity agreement. All main-text state interventions use the corrected implementation, including the synthetic and final-position sweeps (§4.1). Attention knockout does not use state capture. Archived historical development results retain their original sample sizes and provenance. The corrected full-data rerun in Appendix B supersedes the earlier 60-item battery.

Numerics. Log-probabilities on the synthetic control are fp32, because bf16 costs 0.5–2.5 nats on a token deep in the tail. The 8B numbers are bf16, which appears as self-transplant deviations of 0.1–1.1 nats while the accuracy channel remains an exact no-op; accuracy is read first throughout.

B The full-dataset rerun, and what reads the span

The original MedCalc × Qwen3-8B intervention battery was developed on the forty calculators with the most rescued items, sampling four each: 160 of 467 rescued items, and 60 items over 15 calculators after the receiver-never-solves restriction. We reran each of them with every rescued item of every calculator that has at least two (467 items, 48 calculators), changing nothing else. The restriction is stricter on the larger set—a calculator now has to fail on up to twenty items rather than four—so it keeps 135 items over 14 calculators, and we also report the per-item restriction (369 items over 45), which keeps the items whose own receiver fails.

The transplant, the truncation knee between 32 and 64 directions, and the formula quarter's dominance all reproduce. The one arm that moves is the same-family donor, from 0.05 to 0.12, on the 41 items where it is defined.

The depth sweep at one-layer resolution. The subset sweep measured layers 0, 4, 8, 12, 14, 15, 16, 20 and placed the collapse somewhere after 12. At full scale we added 13 and 17–19: recovery is 0.98, 0.98, 0.89, 0.74 at layers 0, 4, 8, 12, then 0.16, 0.16, 0.06, 0.06, 0.07, 0.03, 0.02, 0.03 at 13 through 20 (per-calc; the per-item restriction gives 1.00, 0.98, 0.94, 0.79 then 0.32, 0.34, 0.25, 0.25, 0.23, 0.19, 0.14, 0.17). Layers 13 and 14 are indistinguishable under both restrictions, so the transition is a single layer between 12 and 13 and not the gradual decline a stride-2 sweep suggests.

A continuous dependent variable at the same boundary. Accuracy over a chain of thought is a threshold, and a smooth decline inside the model could present as a step in it. We therefore decode the with-document trajectory once per item and teacher-force that same trajectory under every other condition, reading its mean token log-probability: $q = (\ell_L - \ell_{\text{recv}})/(\ell_{\text{gold}} - \ell_{\text{recv}})$ has the same two anchors as ρ with no decoding threshold anywhere in it ($n = 442$ items on which the document run is correct, 48 calculators). q reads 0.999, 0.989, 0.960, 0.831 at layers 0, 4, 8, 12, then 0.598 [0.550, 0.648] at 13, 0.595 at 14, 0.340 at 16, 0.201 at 20 and 0.129 at 24. Expressed per layer as a fraction of its range, the step at $12 \rightarrow 13$ is 26.8%, against 3.7% per layer over 8–12 and 0.3% from 13 to 14: the discontinuity is present in a readout that cannot be produced by grading. It is smaller than the behavioural step (61% of range), so the grader sharpens the boundary roughly twofold without creating it, and it is spread evenly across the four quarters of the trajectory (0.20, 0.26, 0.31, 0.21) rather than concentrated at the answer, which is the opposite of what a grading artefact predicts. We also tried restricting q to the tokens after the final answer marker and report that it carries no information here: once the full reasoning is teacher-forced, every condition predicts the final number almost perfectly (median ℓ between −0.001 and −0.005 for all of them, and the gold-minus-receiver denominator has median 0.0014, below 0.01 on 59% of items).

Distributed reading and receiver state. Head-level readout from the skill span concentrates in layers 12–21 after normalisation by residual-stream norm. Blocking the 32 highest-ranked heads costs 0.39 of the skill's effect, versus 0.23 for random heads at the same layers (held-out calculators; paired difference 0.154, 95% CI [0.061, 0.251]). Reading is distributed rather than confined to a small circuit. After late span replacement, downstream readout remains 0.86–0.99 of the donor's despite weak behavioural recovery. Earlier receiver question-state updates remain unchanged, motivating the interaction tests in §4.4.

B.1 Focused source-history and path-specific experiments

The new Qwen3-8B/MedCalc experiments fix 124 historical item IDs from 14 calculators before inference and partition them as every fourth ID across four RTX 4090 workers. They use bf16, eager attention, greedy decoding and 1,536 new tokens. Fresh baselines retain 116 donor-correct, receiver-incorrect items, of which 114 also fail without a skill. Both experiments use these same 114 items; all 372 baseline generations agree token-for-token between the two suites. The complete repair grid contains 2,356 generations, the path grid 1,488, and worker identity checks 100. All 48 own-state writebacks are token-exact.

S writes the donor source-span block output at layer L. C replaces only source-position keys and values at layers 0 through L after prefix prefill and before recomputing the final prompt token. SC combines them; SC recutting blocks reader queries to source keys from $L + 1$ onward in that same run. Earlier reader history is unchanged. Reader queries are positions after the source span; source-self queries are positions within it. The path experiments cut the specified edges during both prefill and decoding, preserving the causal mask.

We retain all intervention outcomes, including truncated outputs. For the prespecified SC-minus-S contrast at layer 16, the gain is 38.6 percentage points, with a skill-cluster bootstrap 95% interval of [26.6, 46.5] (10,000 resamples of the 14 calculators, retaining their items). The corresponding joint-minus-additive contrast is 31.6 points [14.1, 41.6]; SC-minus-recut is 35.1 points [23.2, 43.0]. On the 96 items with all three baselines untruncated, SC-minus-S is 38.5 points [25.0, 47.6]. At layer 16, S/SC/recut truncate 4/4/3 of 114 outputs; worst-case reassignment of all truncated S and SC outcomes leaves a 35.1-point repair advantage. At layer 24, SC and recut truncate 3/28 outputs and do not exhibit a reliable recutting loss; this depth is not used to claim independence after repair.

At layer 24, reader and source-self cuts retain 29/114 and 110/114 correct answers. Their loss difference is 71.1 points [53.3, 86.5]. Reader cuts truncate 21 outputs; counting all as correct still leaves retention at 50/114, below source-self retention 110/114 (no truncation). Both contrasts survive worst-case reassignment of truncated outputs. Analysis artifacts retain arm-wise counts, hashes, clustered intervals and truncation bounds.